TRIBHUVAN UNIVERSITY

FACULTY OF HUMANITIES AND SOCIAL SCIENCE

Project Report

on

"Movie Recommendation System Using Hybrid Filtering"

Submitted to

Department of Computer Application

National College of Computer Studies

In partial fulfilment of the requirements of Bachelor's Degree of Computer Application

Submitted By:

Rusha Manandhar

(6-2-551-178-2021)

Under the Supervision of

Sumit Ghising

Date: 15th Shrawan, 2082

Tribhuvan University

Faculty of Humanities and Social Sciences

National College of Computer Studies

Supervisor's Recommendations

I hereby recommend that this project prepared under my supervision by **Rusha Manandhar** entitled "**MovieMagic: The Streaming Aspect of Cinematography**" in partial fulfillment of the requirements for a degree of Bachelor of Computer Application is recommended for the final evaluation.

_____

SIGNATURE

Sumit Ghising

SUPERVISOR

Faculty Member

Department of Computer Application

National College of Computer

Tribhuvan University

Faculty of Humanities and Social Sciences

National College of Computer Studies

Letter of Approval

This is to certify that this project prepared by **Rusha Manandhar** entitled "**MovieMagic: The Streaming Aspect of Cinematography**" in partial fulfillment of the requirements for the degree of Bachelor of Computer Application has been evaluated. In our opinion, it is satisfactory in scope and quality as a project for the required degree.

Signature of Supervisor	**Signature of HOD / Coordinator**
Sumit Ghising Faculty Member Department of Computer Application National College of Computer Studies Paknajol, Kathmandu	Rajan Poudel Faculty Member Department of Computer Application National College of Computer Studies Paknajol, Kathmandu
Signature of Internal Examiner	**Signature of External Examiner**

ABSTRACT

MovieMagic is a movie streaming website designed to help users discover movies they'll love by offering personalized recommendations and localized trends. It does this through a hybrid filtering system—combining content-based filtering (using TMDb's similar movies API) to recommend movies similar to what a user has watched, and collaborative filtering (using the Adjusted Cosine Similarity algorithm) to suggest films liked by users with similar preferences. To further enhance discovery, the platform features location-based trending, which identifies popular movies near the user by calculating distance using the Haversine formula and the browser's built-in location data. Users can also search easily with simple filters based on title, genre, or keyword. All movie information is fetched in real-time from the TMDb API, ensuring updated and reliable content. MovieMagic was built to simplify and personalize the movie-watching experience—making it easier for users to find content they care about, quickly and effortlessly.

Keywords: MovieMagic, personalized recommendations, hybrid filtering, Adjusted Cosine Similarity, TMDb API, location-based trending, movie streaming platform

ACKNOWLEDGEMENT

We would like to extend our heartfelt gratitude to our supervisor, **Mr. Sumit Ghising**, for his invaluable guidance, continuous monitoring, and unwavering motivation throughout all phases of this project. His expertise, constructive feedback, and insightful suggestions were instrumental in shaping the direction and success of this endeavour. We deeply appreciate his patience, support, and the inspiration he provided, which helped us overcome various challenges and achieve our goals. We also wish to express our sincere appreciation to all other supervisors, professors, and mentors who offered their time, knowledge, and encouragement. Their collective support and advice were crucial in refining our approach and enhancing the quality of our work. Additionally, we are grateful to our friends, peers, and family members for their constant encouragement, moral support, and willingness to assist whenever needed. This project would not have been possible without the collaborative efforts and contributions of everyone involved, and we are truly thankful for their role in making it a success.

Rusha Manandhar

TABLE OF CONTENT

LIST OF ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
PHP	Hypertext Preprocessor
SQL	Structured Query Language
TMDb	The Movie Database

LIST OF FIGURES

LIST OF TABLES

Chapter 1 :
INTRODUCION

1.1 INTRODUCTION

In today's digital age, the way people discover and consume movies has undergone a significant transformation. Streaming platforms have emerged as the dominant source of entertainment, offering vast libraries of content accessible at any time and from any location. While this has made it easier than ever to watch films and TV shows, it has also introduced a new challenge—navigating through an overwhelming number of choices to find content that genuinely aligns with individual preferences.

MovieMagic addresses this challenge by offering a smart, personalized movie streaming experience powered by advanced recommendation techniques and real-time data integration. The platform is designed to go beyond traditional search functionality by providing users with tailored content suggestions that resonate with their viewing history, tastes, and preferences.

At the core of MovieMagic lies a **hybrid recommendation system** that blends two powerful techniques: content-based filtering and collaborative filtering. The content-based filtering mechanism utilizes the TMDb (The Movie Database) API to suggest movies that are similar to those the user has previously watched. This ensures that users receive recommendations closely aligned with their existing interests. On the other hand, collaborative filtering uses the Adjusted Cosine Similarity algorithm to analyse user behavior and recommend movies based on the preferences of users with similar viewing patterns. This dual approach significantly enhances the relevance and variety of suggested content.

In addition to personalized recommendations, MovieMagic introduces **location-based trending** as a unique feature to help users discover what's popular in their geographical area. By leveraging the Haversine formula along with browser-based location detection, the platform can identify trending movies in the user's vicinity, making the discovery process more contextually relevant and engaging.

The platform also emphasizes user experience with a **clean, intuitive interface** and **easy-to-use search filters**, allowing users to find and explore movies effortlessly. All movie

data is sourced from the reliable TMDb API, ensuring up-to-date and accurate information. From user authentication to personalized dashboards, every feature is built with the goal of making movie discovery more interactive, meaningful, and enjoyable.

In summary, MovieMagic redefines the streaming experience by combining intelligent recommendations, localized insights, and a user-friendly design. It not only simplifies the process of finding content but also transforms it into a more personal and engaging journey for every movie lover.

1.2 Problem Statement

In today's fast-growing digital world, users have access to thousands of movies across various streaming platforms, making it increasingly difficult to find content that matches their personal taste. As a result, users often spend more time searching than watching, leading to a frustrating and time-consuming experience. Additionally, most recommendation systems fail to consider both user preferences and regional trends. MovieMagic is developed to solve this problem by providing smart, real-time, and location-aware movie recommendations through hybrid filtering techniques and real-time data integration.

1.3 Objective

The main objective of MovieMagic is:

● To develop a movie streaming platform that suggests movies to users by analysing their watch history and reviews using a combination of content-based and collaborative filtering techniques.

● To use the Haversine formula to add location-based features so users can find popular movies that people are watching near them.

1.4 Scope and Limitation

1.4.1 Scope

The domain of this project is to provide users with a smooth and personalized experience for discovering movies and TV shows. It uses a hybrid recommendation system that combines 70% content-based filtering and 30% collaborative filtering,

based on users' watch history, ratings, and reviews to suggest content they may enjoy. The platform retrieves real-time and accurate movie details using the TMDB API, keeping users updated with the latest releases. It also highlights trending movies based on the user's location by analysing local view counts, helping users explore popular films in their area. With a simple search feature and easy-to-use interface, MovieMagic focuses on making the process of finding and exploring movies more enjoyable and relevant for every user.

1.4.2 Limitation

While MovieMagic offers personalized recommendations and location-based trending movies, its effectiveness depends on active user participation. If users do not regularly watch, rate, or review movies, the recommendation system may not provide highly accurate or relevant suggestions. Additionally, the platform relies on data from the TMDB API, so any limitations or downtime in that service could affect the availability of real-time movie updates. Since the search feature is basic, users might find it harder to locate specific movies if they don't know exact titles or details. Finally, MovieMagic focuses on helping users discover movies but does not include direct streaming or downloading options, which may limit the overall viewing experience.

1.5 Development Methodology

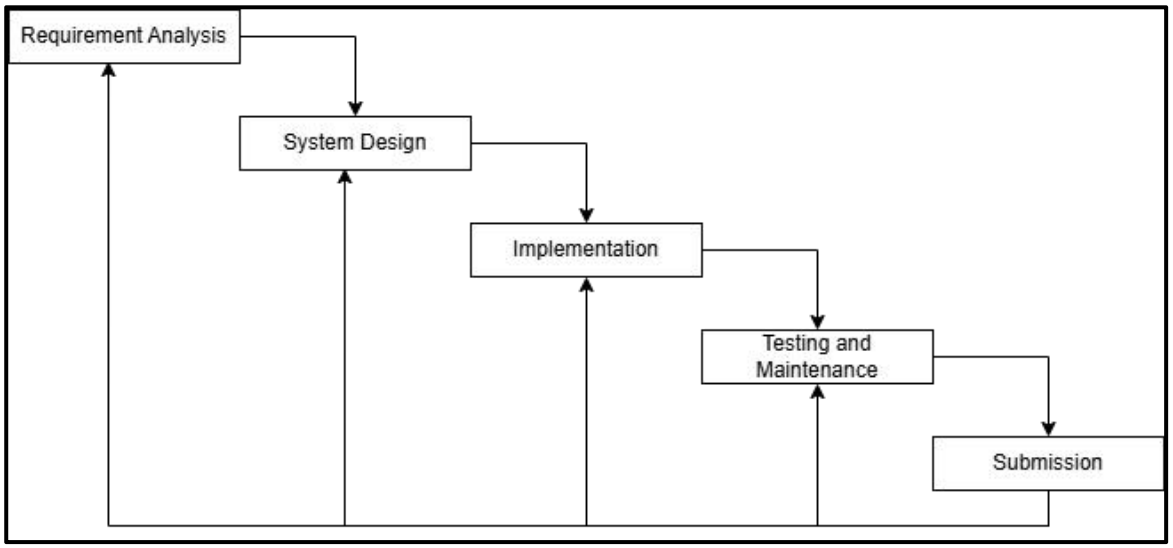


Figure 1.1:Waterfall Model

We used the Waterfall Model for this project because it offers a clear and structured development process with well-defined stages. It also ensures proper documentation and

quality checks at each step. This model was a good fit for us since the project scope was clear from the beginning, and only a few changes were expected during development.

1.6 Report Organization

Chapter 1: Introduction

This chapter provides an overview of the project, including the problem it aims to solve, the project objectives, scope and limitations, and development methodology of the project.

Chapter 2: Background Study and Literature Review

This chapter focuses on study of existing systems and review of different articles studied. It also explores the underlying concepts and principles that informed the development of the system.

Chapter 3: System Analysis and Design

This chapter covers the project's requirements analysis, feasibility assessment, and design. System Models as Data Model, Process Model and Flowcharts, functionality analysis and System Designs as Architecture Design, Database Schema Design, Interface Design and Physical DFD are all included.

Chapter 4: Implementation and Testing

This chapter provides details on the project's implementation, the software and tools used, and the types of testing conducted.

Chapter 5: Conclusion and Future Recommendations

This chapter discusses the project's potential outcomes and summarizes the main findings. It also provides recommendations for future work

Chapter 2 :
BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

With the rapid rise of digital media, people now mainly watch movies and shows through online streaming platforms like Netflix and Amazon Prime. While this offers many choices, it also creates a problem—too much content makes it hard for users to find something they'll actually enjoy. MovieMagic aims to solve this by providing smart and personalized movie recommendations. It uses a hybrid system that combines two methods: content-based filtering (70%), which suggests movies similar to ones the user has already liked using the TMDb API, and collaborative filtering (30%), which recommends movies based on what similar users enjoy using the Adjusted Cosine Similarity method. What makes MovieMagic stand out is its location-based feature that shows trending movies in the user's area using the Haversine formula and browser location. The platform also offers a clean and user-friendly interface, smart search options, and reliable movie data, all designed to make finding the right movie faster, easier, and more enjoyable. Overall, MovieMagic uses smart techniques to fix the common problems of today's recommendation systems and helps users find the best movies for them.

2.2 Literature Review

The integration of content-based and collaborative filtering techniques in hybrid recommendation systems has been shown to enhance recommendation accuracy and user satisfaction. Geetha et al. proposed a movie recommendation system that combines these two approaches to provide more precise suggestions, effectively addressing challenges such as the cold-start problem and data sparsity [1].

Graph-based hybrid models have been explored to further improve recommendation quality, particularly in cold-start scenarios. Darban and Valipour introduced a Graph-based Hybrid Recommendation System (GHRS) that incorporates user demographics and location information, demonstrating significant improvements in recommendation accuracy and effectively mitigating the cold-start issue [2].

To tackle the limitations of traditional collaborative filtering in sparse data environments, Qian et al. developed Attribute Graph Neural Networks (AGNN). This approach

leverages user and item attributes to learn embeddings, enabling effective recommendations even when interaction data is limited or nonexistent [3].

Another innovative solution to the cold-start problem is the GPatch framework proposed by Chen et al., which employs graph neural networks to model both warm and cold-start users/items. By constructing correlated patching networks, GPatch maintains high recommendation performance across different user/item scenarios [4].

Feature weighting techniques have also been utilized to enhance hybrid recommendation systems. Bernardis et al. presented a model that learns the importance of item features from collaborative information, resulting in improved recommendations for cold-start items by effectively combining content-based and collaborative filtering methods [5].

Incorporating artificial neural networks into hybrid recommendation systems has shown promise in optimizing performance. A study introduced a system that combines content-based filtering, collaborative prediction, and self-organizing maps, achieving higher accuracy and precision compared to traditional methods [6].

Furthermore, Widayanti et al. demonstrated that a hybrid approach integrating collaborative filtering and content-based filtering can effectively address the cold-start problem and enhance recommendation diversity. Their system showed improved performance by leveraging the strengths of both techniques [7].

Li et al. explored a hybrid algorithm that combines content and collaborative filtering to optimize recommendation systems. Their approach addresses issues such as data sparsity and the cold-start problem, ensuring better recommendation quality and system performance [8].

A practical example of a recommendation system is **FilmQuest**, an open-source platform that implements a movie search and recommendation engine using modern web technologies and integrates TMDb APIs to fetch relevant movie data and recommendations. Its architecture emphasizes modular design and open contributions, serving as a foundation for personalizing movie discovery interfaces [9].

Similarly, **MovieVault** offers a real-time movie discovery platform built with Next.js 14 and server-side rendering. It combines filtering, search optimization, and dynamic UI

updates to deliver a smooth user experience. By incorporating API data fetching and personalized search, it aligns with modern approaches in scalable streaming solutions [10].

Chapter 3 :

SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

3.1.1 Requirement Analysis

Requirements will be collected through personal evaluation of different existing systems, along with suggestions from mentors, and supervisors.

3.1.1.1 Functional Requirements

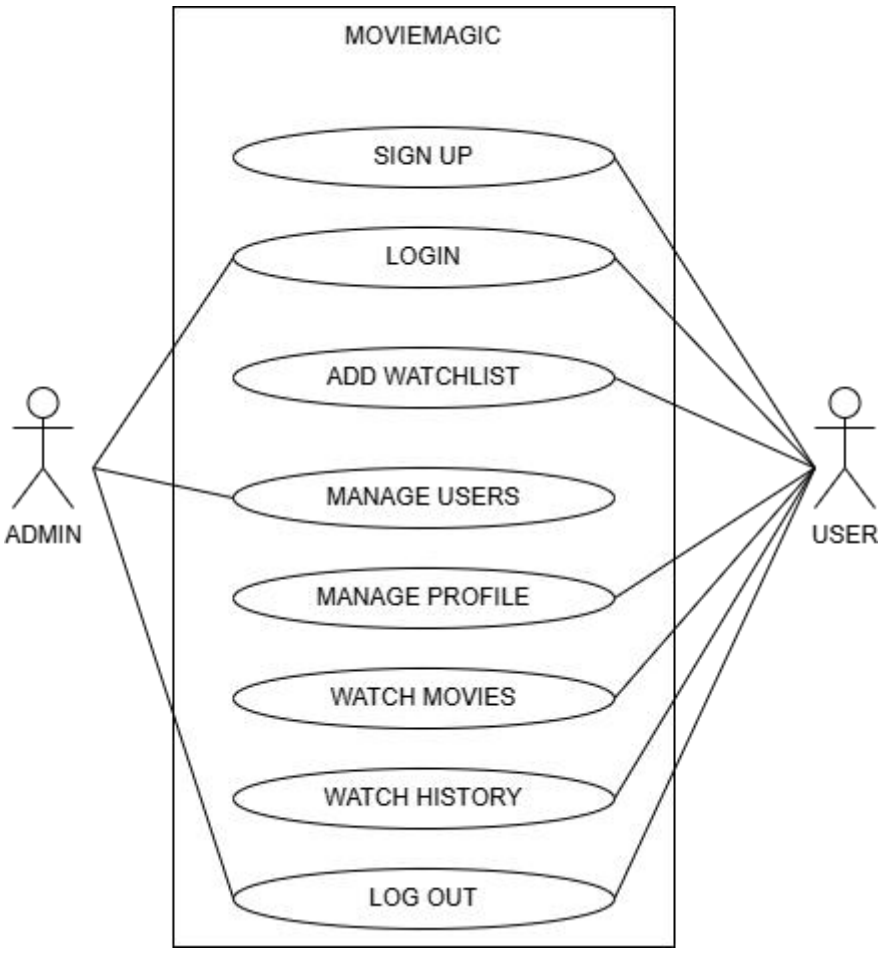


Figure 3.1: Use Case Diagram of MovieMagic

The use case diagram in previous page specifies the basic operations a user can perform in the proposed system

- A new account creation feature with a username, email and password is required.
- With their username and password, users must be able to access their accounts.
- Users with wrong username will neither be validated nor allowed to login.
- Users with correct credentials must be able to perform CRUD (Creation, Retrieve, Update and Delete) operation on the entries (Profile Info and Watch History).
- The site should be able to stream movies/TV shows from the catalogue, bookmark movies/TV shows for later viewing, auto-saves progress in watch history.
- Each watch history lists all watched content with timestamps.
- The site should include option to remove entries of watch history.
- Users should be able securely ends the user session.

3.1.1.2 Non-Functional Requirements

- It works on phones, tablets, and computers.
- User data are kept safe and private at all times.
- The site is easy to use and understand for everyone.
- It works on popular web browsers like Chrome and Firefox.
- The system is stable and do not crash often.
- The website displays clear error messages when something goes wrong
- The website tries to load results as quickly as possible

3.1.2 Feasibility Study

The system is evaluated for future development with a set of constraints. The feasibility study is done regarding the available technologies, time constraints, area of application, cost of deployment and upkeep, and future possibilities of the project.

3.1.2.1 Technical Feasibility

The technical feasibility of the project has been evaluated by considering the use of widely adopted and reliable tools and technologies. The frontend is developed using HTML, CSS, and JavaScript, which contribute to a user-friendly and interactive interface. The backend is powered by the Apache Web Server, MySQL database, and PHP, effectively managing data and server-side operations. Additionally, the project integrates the free TMDb API to fetch real-time movie information and utilizes the browser’s location services to display content based on the user's geographic area. These choices

make the project straightforward to build, operate, and maintain within a limited timeframe and budget.

3.1.2.2 Operational Feasibility

The operational feasibility of the system has been assessed and found to be strong, as it is user-friendly and integrates well into a typical working environment. Users can easily register, log in, search for movies, and receive personalized recommendations, including location-based trending suggestions. The interface is simple and intuitive, allowing even users with minimal technical knowledge to navigate the platform without difficulty. From the administrator's perspective, managing user data is straightforward, making it easy to maintain the system and ensure smooth daily operations.

3.1.2.3 Economic Feasibility

From an economic perspective, the project is highly feasible as it relies on open-source and free technologies such as HTML, CSS, JavaScript, PHP, MySQL, and the TMDb API, thereby eliminating licensing costs. Additionally, development tools like Visual Studio Code, GitHub, and the Brave browser are freely available, further reducing expenses. Since the platform can be hosted on low-cost servers, the overall cost of development and maintenance remains minimal and manageable.

3.1.3 Data Modelling: Entity Relation Diagram

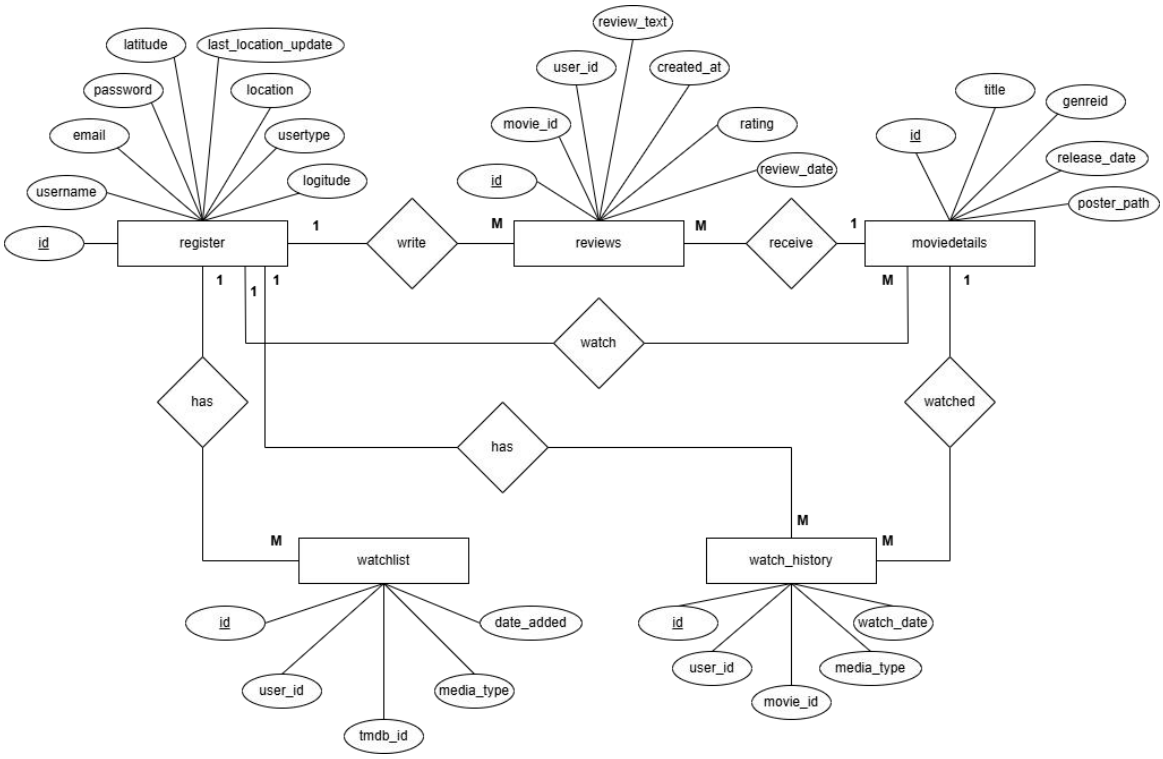


Figure 3.2: Entity Relation Diagram of MovieMagic

The above table represents the following relation:

- One-to-many relationship between "**Register"** and "**Reviews"** entity where one user can write multiple reviews.
- One-to-many relationship between "**Register"** and "**Watchlist"** entity where one user can save multiple movies/TV shows to their watchlist.
- One-to-many relationship between "**Register"** and **"Watch_history"** entity where one user can have multiple entries in their watch history.
- One-to-many relationship between "**Moviedetails"** and **"Reviews"** entity where one movie can receive multiple reviews from users.
- One-to-many relationship between "**Moviedetails"** and **"Watch_history"** entity where one movie can be watched by multiple users.
- One-to-many relationship between "**Moviedetails"** and **"Register"** entity where a user can watch many movies.

3.1.4 Data Modelling: Data Flow Diagram

i. DFD Level 0

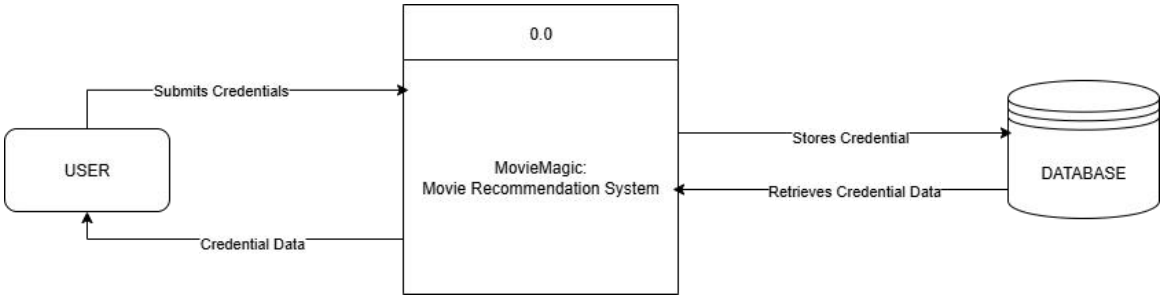


Figure 3.3: DFD Level 0 of MovieMagic

ii. DFD Level 1

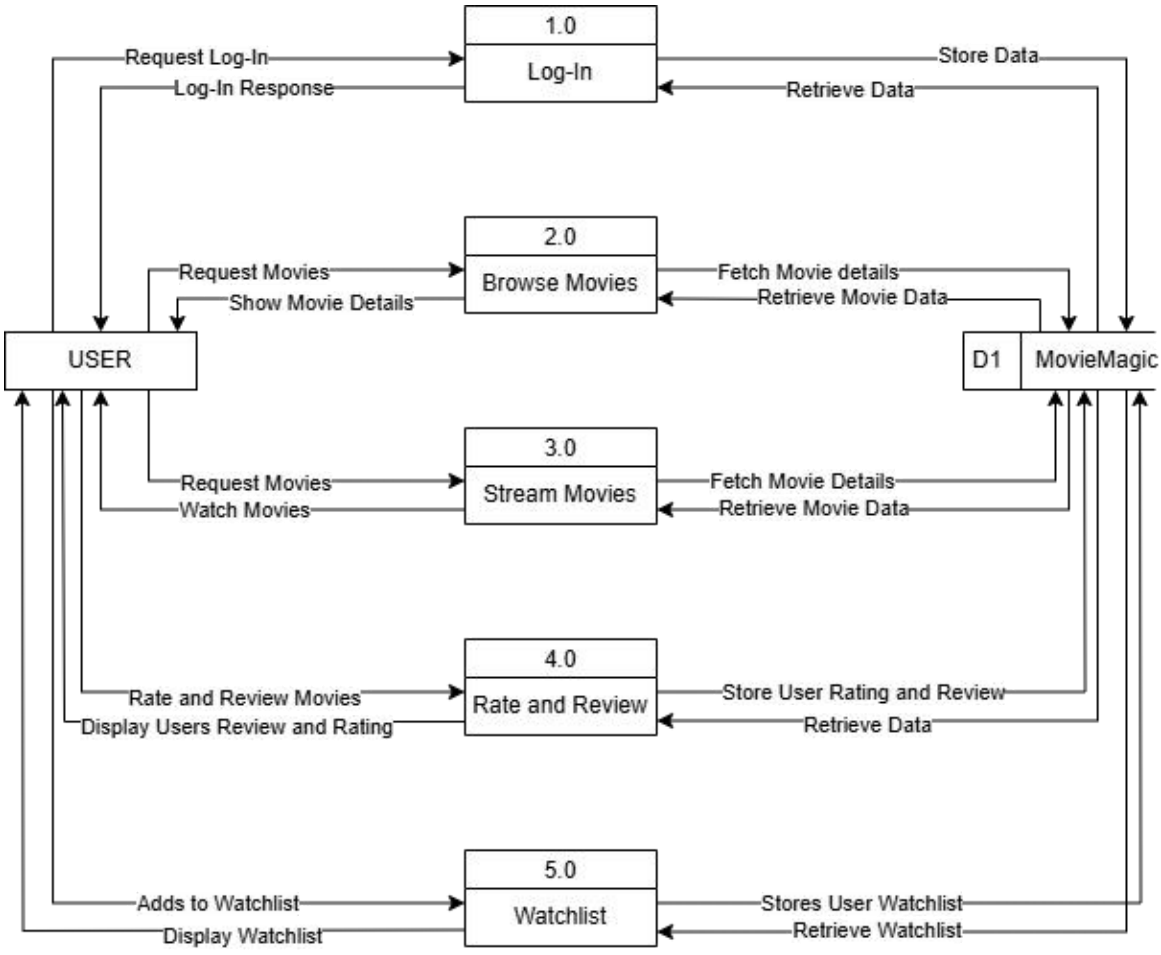


Figure 3.4: DFD Level 1 of MovieMagic

The data flow in the "MovieMagic" system includes user register or log in, storing their details in the database. They can browse movies, which pulls data from the external source like TMDB, and save favourites to their watchlist. When they watch or rate a movie, it updates their history and reviews. Admins can manage all users. Everything is securely stored in a MySQL database.

3.2 System Design

3.2.1 Architecture Design

The **two-tier architecture** has been used for this project. In this architecture, the application is divided into two main layers: the **client layer** (presentation) and the **server layer** (which combines both application logic and data storage). This model enables direct communication between the client and the server, reducing complexity and providing faster interaction for smaller-scale applications.

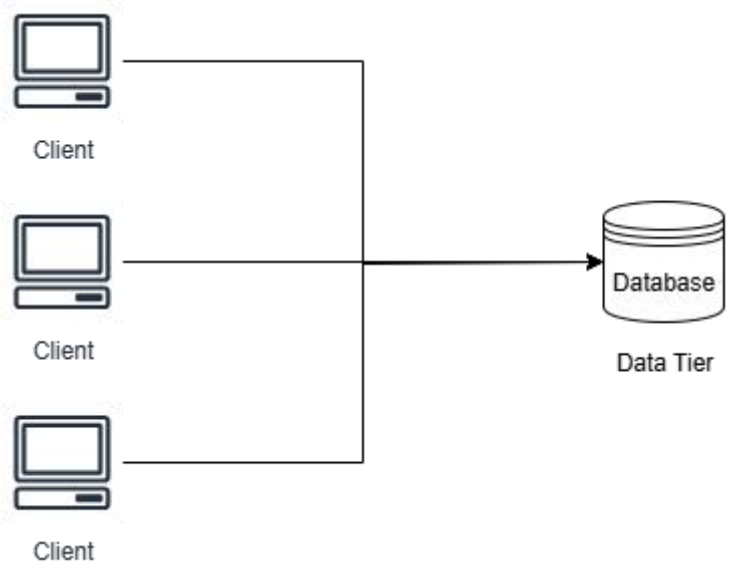


Figure 3.5: Two Tier Architecture

Two-Tier Architecture has the following layers.

- **Client Layer:** The Presentation Layer forms the front-end of MovieMagic and is developed using HTML, CSS, and JavaScript. It enables users to interact with the application by browsing content, searching for movies, logging in, viewing personalized recommendations, and managing their watchlists. It handles user inputs and displays dynamic content retrieved from the server, supporting features such as real-time search and intuitive navigation.
- **Server Layer:** The Server Layer combines both the application logic and database operations. Developed using PHP and MySQL, it processes client requests by handling user authentication, watch history, watchlists, and search queries. It communicates with the TMDb API to fetch real-time movie data and generates personalized recommendations using hybrid filtering (70% content-based and 30% collaborative). The server also manages data storage in a MySQL

database hosted on an Apache Web Server, storing user information, movie details, ratings, reviews, and activity logs.

Following are the reasons behind choosing two-tier architecture:

- It simplifies the development and deployment process by minimizing the number of layers.
- Direct interaction between the client and server ensures faster response times and improved performance for lightweight applications.
- It is suitable for applications like MovieMagic that do not require complex enterprise-level scaling or distributed components.
- Maintenance is easier for smaller teams, and development is faster due to fewer architectural components.

3.2.2 Database Schema Design

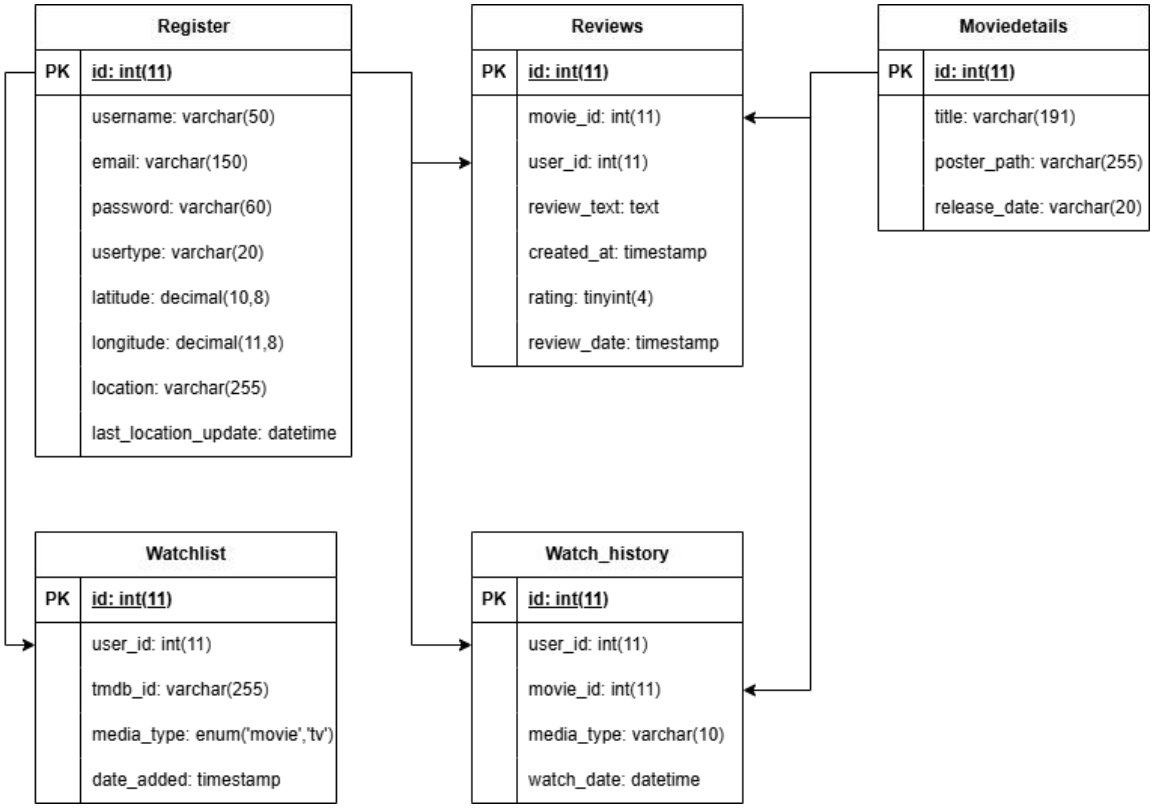


Figure 3.6: Database Schema of MovieMagic

The database schema design for MovieMagic includes the following tables:

1. **register**: This table stores user information, including the ‘id’, ‘username’, ‘email’, ‘password’, ‘latitude’, ‘longitude’, ‘location’ and `last_location_update’ datetime.
2. **reviews**: This table store the information about user generated ratings and text reviews, including the ‘user_id’ and ‘movie_id’ as foreign keys.
3. **moviedetails**: This table stores core movie/TV show metadata (title, release date, poster URL). It includes relationships like; 1-to-many with reviews (movies receive multiple reviews). 1-to-many with watch_history (movies appear in multiple view records).
4. **watchlist:** This table tracks user-saved "to watch" items, including user_id and tmdb_id(external tmdb api reference) as it’s foreign keys.
5. **watch_history**: This table logs view content with timestamps. The user_id and movie_id fields acts as a foreign key constraints

3.2.3 Interface Diagram

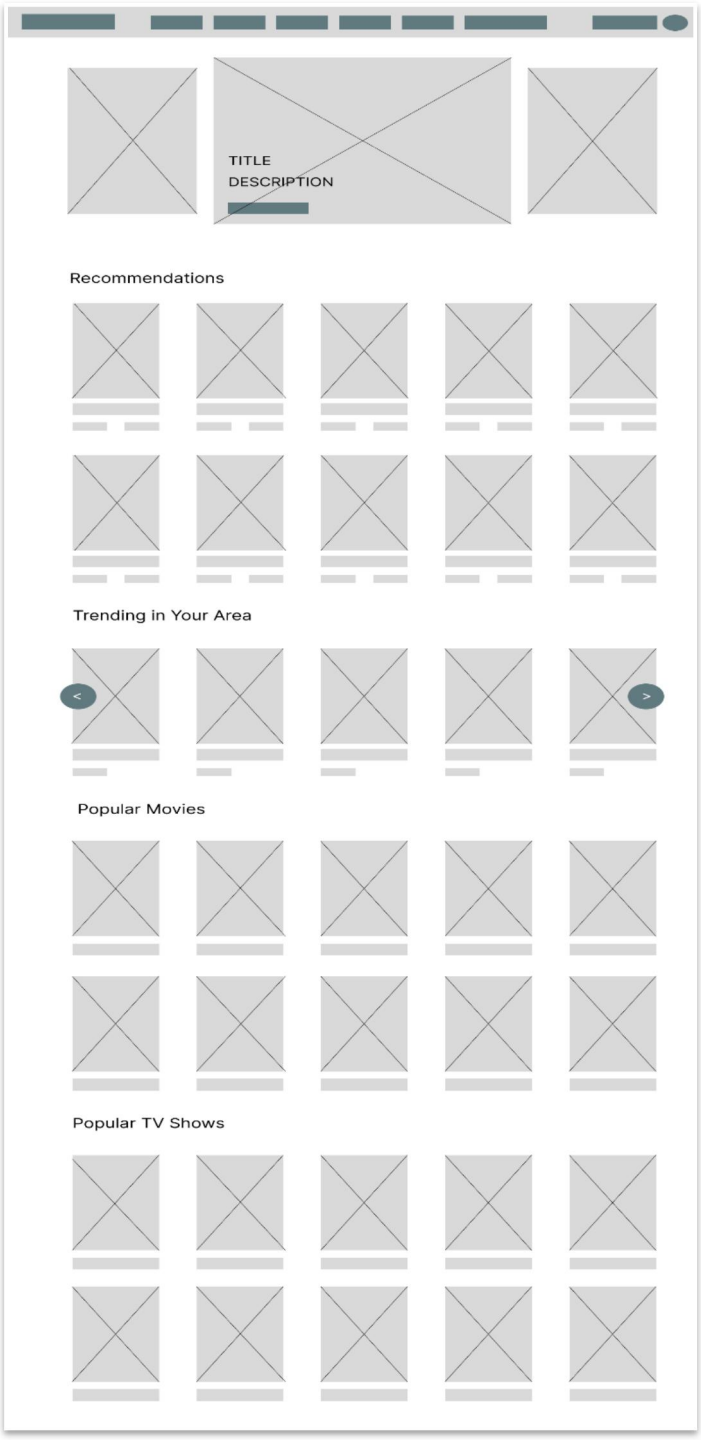


Figure 3.7: Interface Diagram of MovieMagic

The interface shows a streaming platform layout with two main parts. At the top is the navigation bar (header) containing the app logo, menu options, search box, and user profile. Below this is the main content area which displays the recommendations section, trending in your area, popular movies and popular tv-shows. Here in treading in your area you'll find swipeable rows of content represented by rectangular boxes: those with

crossed lines show movie/TV show images, while other boxes display titles and media type labels. The clean layout makes it easy to browse different options, with visual thumbnails and clear text information helping users quickly find something to watch.

3.2.4 Physical DFD

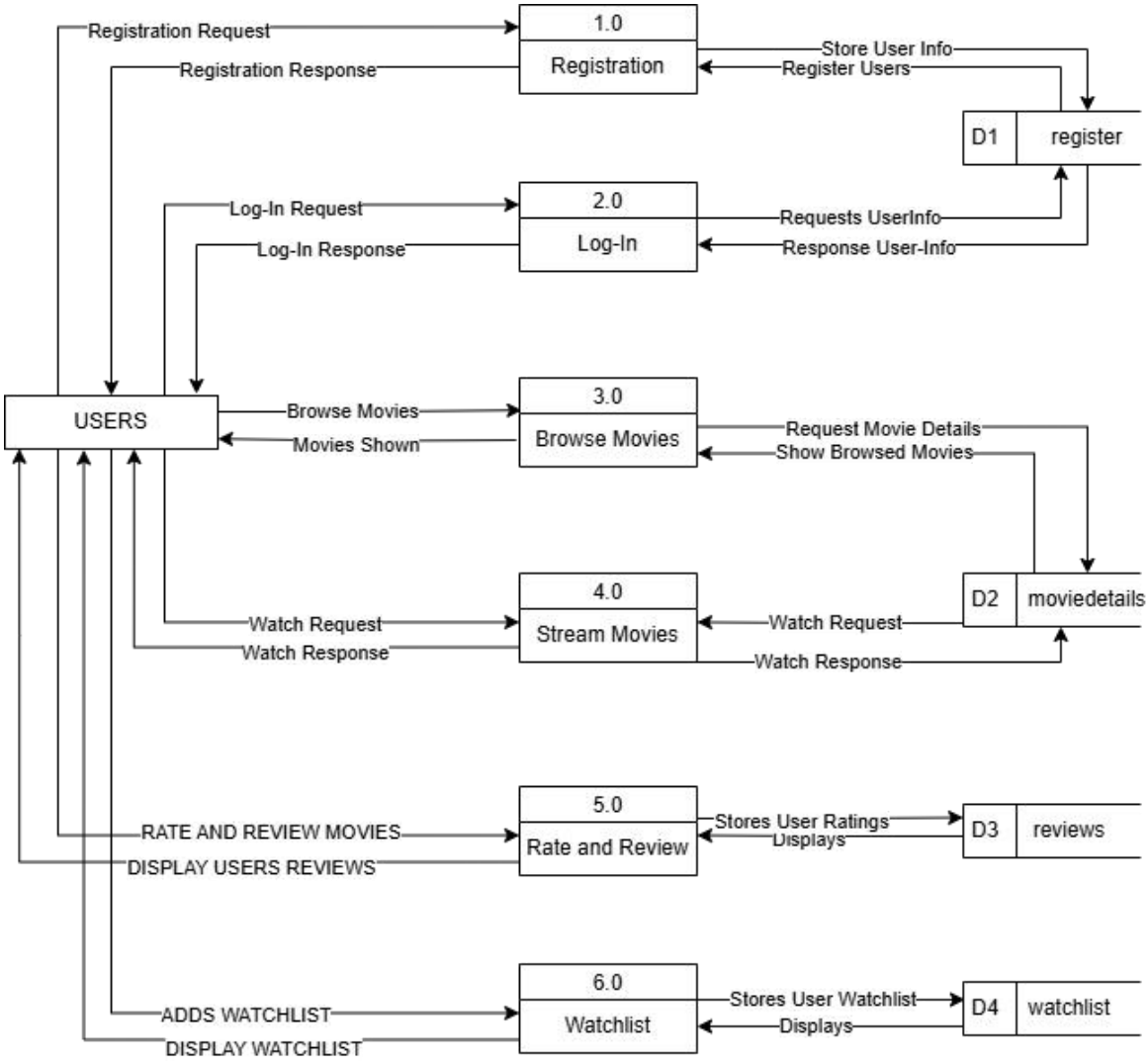


Figure 3.8: Physical DFD of MovieMagic

The above figure is a Physical DFD that visualizes user's activity, and the flow of the data between the user, and MySQL database. The user performs activities as logging in, searching, watching, rating, updating, checking history, and inserting favorites to their watchlist.

3.2.5 Algorithm Details

The MovieMagic platform implements a hybrid recommendation system that combines content-based and collaborative filtering approaches to provide personalized movie and TV show recommendations to users.

The recommendation comprises the following steps:

1. Hybrid Recommendation Approach: The system uses a weighted combination of two recommendation methods: Content-based filtering (70% weight) and Collaborative filtering (30% weight)
2. Content-based Filtering:
    - The first round of content-based filtering involves tracking users watch history, records movie IDs and media type and stores user rating for watched content
    - The second round uses TMDb API to fetch similar content for each watched item, considers movie metadata and attributes, then normalizes scores to a 0-1 scale
    - The third-round scores formula (score=0.7 *(vote_average/10))
3. Collaborative Filtering:
    - The first round of Collaborative Filtering involves using adjusted cosine similarity which requires minimum of 2 common rated movies between users with Similarity threshold: 0.3
    - The second round checks the similarity using Similarity formula (similarity = (pSum - (sum1 * sum2 / n)) / sqrt((sum1Sq - pow(sum1, 2) / n) * (sum2Sq - pow(sum2, 2) / n)))
        - Where:
            - pSum: Sum of product of ratings
            - sum1, sum2: Sum of ratings for each user
            - sum1Sq, sum2Sq: Sum of squared ratings
            - n: Number of common movies
    - The third round generates the recommendation according to the Aggregates ratings from similar users which Requires minimum of 2 similar users and then it Normalizes scores to 0-1 scale using scoring formula (0.3 * (avg_rating / 5)).
4. Recommendation Process Flow:
    - It initially verifies if the user has watched any content or not through users watch history and ratings.
    - For, Content-based filtering it fetches similar content for each watched item and excludes already watched content. Then it calculates content-based scores.

- For, Collaborative filtering it firstly, identifies similar users based on rating patterns and collects highly-rated content from similar users and excludes user's watched content. Then at the end it calculates collaborative scores

5. Output: the output of Hybrid Filtering is it combines both recommendation type sort by combined score and returns top 12 recommendations.

In summary, The Hybrid Approach provides a robust foundation for personalized content discovery. The combination of content-based and collaborative filtering methods ensures diverse and relevant recommendations while maintaining scalability and performance.

Chapter 4 :
IMPLEMENTATION AND TESTING

4.1 Implementation

This section lists all the tools and technologies used to build the *MovieMagic* movie recommendation platform. These tools were chosen because they are reliable, easy to use, and work well together for creating a dynamic website that gives real-time movie updates and personalized recommendations.

1. CASE Tools

- **XAMPP:** Provided a complete local development environment, including Apache server and MySQL, to test and run the application offline during development.
- **Canva:** Used to design user interface mockups and visual documentation for the system layout and structure.
- **Draw.io:** Employed for creating detailed system design diagrams such as DFDs, ER diagrams, and architecture flowcharts to visualize and document the system's structure and processes.
- **Visual Studio Code (VS Code):** The primary code editor used throughout development, offering a flexible environment with useful extensions for PHP, MySQL, and web technologies.

2. Programming Languages and Development Tools

- **HTML:** Provided the structural foundation for web pages, organizing content in a semantic and accessible manner.
- **CSS:** Responsible for styling the frontend, enhancing visual appeal and layout consistency across the platform.
- **JavaScript:** Enabled interactivity on the client side, powering features like the basic search function and dynamic UI behaviors.
- **PHP:** Handled server-side logic including user authentication, session control, form processing, and integration with the TMDb API.

3. Database Platform

- **MariaDb:** A relational database system used to store and manage all backend data including user accounts, watch history, favorites, and movie metadata. It ensured fast and efficient querying and data retrieval.

4.1.1 Implementation details of modules

In this section, we provide a detailed description of the procedures, functions, classes, and methods used for the algorithms.

Module 1: Location-Based Content Discovery Module

The location-based content discovery module in MovieMagic helps identify users who are geographically close to each other by using the Haversine formula. It retrieves the current user's latitude and longitude from the database, then compares this with the coordinates of other users. If another user is within a 50 km radius, their ID is stored in a list of nearby users. This feature can be used to personalize content, enable local trends, or build community-based recommendations.

Implementation:

```
// Location Data Retrieval
$location_query = "SELECT latitude, longitude FROM register WHERE id = ?";
$location_stmt = $connection->prepare($location_query);
$location_stmt->bind_param("i", $user_id);
$location_stmt->execute();
$location_result = $location_stmt->get_result();
$user_location = $location_result->fetch_assoc();

// Haversine Formula Implementation
function calculateDistance($lat1, $lon1, $lat2, $lon2) {
    $earthRadius = 6371; // Earth's radius in kilometers

    $lat1 = deg2rad($lat1);
    $lon1 = deg2rad($lon1);
    $lat2 = deg2rad($lat2);
    $lon2 = deg2rad($lon2);

    $dlat = $lat2 - $lat1;
    $dlon = $lon2 - $lon1;

    $a = sin($dlat/2) * sin($dlat/2) + cos($lat1) * cos($lat2) * sin($dlon/2) * sin($dlon/2);
    $c = 2 * atan2(sqrt($a), sqrt(1-$a));
```

```
    $distance = $earthRadius * $c;

    return $distance;
}

// Nearby Users Detection
$nearby_users = [];
if ($user_location && $user_location['latitude'] && $user_location['longitude']) {
    $users_query = "SELECT id, latitude, longitude FROM register WHERE latitude IS
NOT NULL AND longitude IS NOT NULL AND id != ?";
    $users_stmt = $connection->prepare($users_query);
    $users_stmt->bind_param("i", $user_id);
    $users_stmt->execute();
    $users_result = $users_stmt->get_result();

    while ($user = $users_result->fetch_assoc()) {
        $distance = calculateDistance(
            $user_location['latitude'],
            $user_location['longitude'],
            $user['latitude'],
            $user['longitude']
        );

        if ($distance <= 50) { // Within 50km
            $nearby_users[] = $user['id'];
        }
    }
}
```

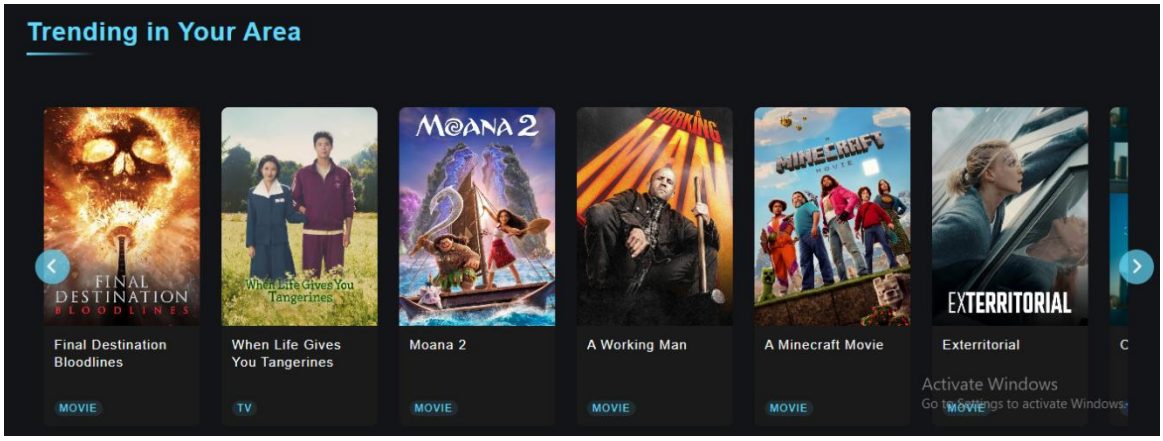


Figure 4.1: Location-Based Content Discovery Module

Module 2: Hybrid Recommendation Engine Module

This recommendation module in MovieMagic combines content-based and collaborative filtering to suggest movies. Content-based filtering (70% weight) fetches similar movies from TMDb based on what the user has already watched. Collaborative filtering (30% weight) compares the user's ratings with those of other users to find similar taste patterns and recommend movies accordingly. The system blends both methods for personalized and relevant suggestions.

Implementation

```
// Content-Based Recommendations (70% weight)
$content_recommendations = [];
foreach ($watched_movies as $watched) {
    $similar_url =
"$tmdb_base_url/{$watched['media_type']}/{$watched['id']}/similar?api_key=$tmdb_api
_key";
    $similar_data = json_decode(file_get_contents($similar_url), true);

    if (isset($similar_data['results'])) {
        foreach ($similar_data['results'] as $movie) {
            if (!in_array($movie['id'], $watched_ids)) {
                $key = $movie['id'] . '_' . $watched['media_type'];
                $content_recommendations[$key] = [
                    'id' => $movie['id'],
                    'title' => $movie['title'] ?? $movie['name'],
                    'poster_path' => $movie['poster_path'],
                    'vote_average' => $movie['vote_average'],
                    'media_type' => $watched['media_type'],
                    'score' => 0.7 * ($movie['vote_average'] / 10)
                ];
            }
        }
    }
}
```

```
// Collaborative Filtering (30% weight)
if (count($user_ratings) >= 3) {
    $similar_users = [];
    foreach ($other_users as $other_user) {
        $common_movies = array_intersect(
            array_keys($user_ratings),
            array_keys($other_ratings)
        );

        if (count($common_movies) >= 2) {
            $similarity = calculateUserSimilarity(
                $user_ratings,
                $other_ratings,
                $common_movies
            );

            if ($similarity > 0.3) {
                $similar_users[$other_user_id] = $similarity;
            }
        }
    }
}
```

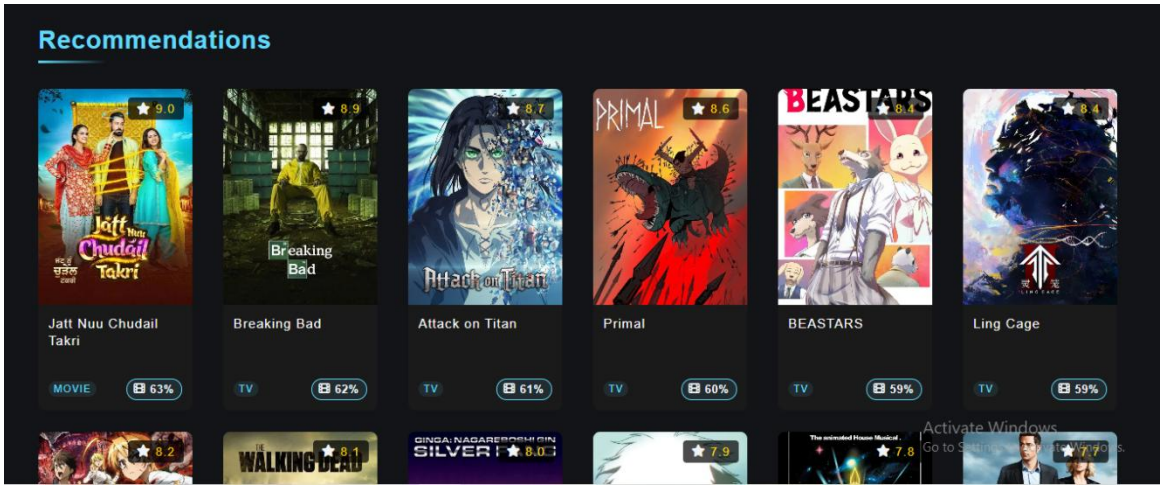


Figure 4.2: Hybrid Recommendation Engine Module

4.2 Testing

4.2.1 Test Cases for Unit Testing

Project Name: MovieMagic

i. Unit Testing of the Registration in MovieMagic

Table 4.1: Unit Testing of the Registration in MovieMagic

Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1. Navigate to registration page 2. Fill form 3. Submit	Username: movielover Email: love@movie.com Password: Movie@123 Re-password: Movie@123	Redirect to login page	Redirect to login page	Pass
1. Navigate to registration page 2. Submit Empty Form	Username: Email: Password: Re-password:	Displays "Please fill out this fields"	Displays "Please fill out this fields"	Pass

ii. Unit Testing of the Login in MovieMagic

Table 4.2: Unit Testing of the Login in MovieMagic

Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1Navigate to login page 2. Enter credentials 3. Submit	Username: movielover Password: Movie@123	Redirect to homepage	User navigated to homepage	Pass
1. Submit with invalid credentials	Username: wrong Password: wrong	Show error "Invalid Username or Password"	Error displayed	Pass

iii. Unit Testing of the Search Functionality in MovieMagic

Table 4.3: Unit Testing of the Search Functionality in MovieMagic

Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1. Enter a valid movie name in search bar 2. Press Search	Query: Inception	Show search results	Search results shown	Pass

1. Enter random characters 2. Press Search	Query: xyzabc123	Show “No results found”	Proper message displayed	Pass

iv. Unit Testing of the Recommendation Feature in MovieMagic

Table 4.4: Unit Testing of the Recommendation Feature in MovieMagic

Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1. Watch 3+ movies 2. Go to recommendation page	Watched IDs: 123, 456, 789	Show relevant movie recommendations	Recommendations displayed	Pass
1. Login as a new user 2. Visit recommendation section		Show trending or default popular movies	Trending list shown	Pass

v. Unit Testing of the Logout Feature in MovieMagic

Table 4.5: Unit Testing of the Logout Feature in MovieMagic

Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1. Click on Logout		Redirect to login page	User logged out	Pass

4.2.2 Test Cases for System Testing

i. System Testing of the MovieMagic (End-to-End)

Table 4.6: System Testing of the MovieMagic

Test Description	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
Registration	Username: moviefan Email: fan@movie.com	Redirect to login page	Success	Pass

	Password: MovieFan@123			
Login	Username: moviefan Password: MovieFan@123	Redirect to homepage	Success	Pass
Search Movie	Query: Avatar	Display matching result	Success	Pass
Watch movie (track watched status)	Movie ID: 2023	Movie marked as watched	Tracked	Pass
Generate Recommendations	Based on watched history	Show personalized list	Shown	Pass
Logout		Redirect to login Page	Success	Pass

Chapter 5 :
CONCLUSION AND FUTURE RECOMMENDATION

5.1 Conclusion

MovieMagic provides a smooth and secure movie streaming experience with features like user authentication, personalized recommendations and search optimization. Its robust validation and security modules ensure data integrity and user safety. Thorough testing confirms the system's reliability, making MovieMagic a well-rounded platform with strong potential for future enhancements.

5.2 Lesson Learnt / Outcome

From this project, I learned about importance of time management while doing a project, how the coding structure (i.e. the folders structure) is maintained and how programming is done on the projects. I also learned to build the diagrams that are needed for the system. I learned many things which can lead us to do any work related to projects further.

5.3 Future Recommendations

To further enhance the MovieMagic platform, we recommend integrating user-centric features that improve accessibility, engagement, and personalization. Introducing real-time collaborative rooms or "watch party" functionality could add a social dimension to the viewing experience, allowing users to enjoy movies together from different locations. Expanding support for multiple languages and subtitles would make the platform more inclusive and accessible to a global audience. Additionally, implementing voice search and playback control through speech recognition would offer a hands-free and user-friendly interface, especially beneficial for mobile and smart TV users. Personalized notifications about trending content, upcoming releases, or tailored movie suggestions could further boost user retention by keeping audiences engaged with timely and relevant updates.

REFERENCE

[1] G. Geetha, M. Safa, C. Fancy, and D. Saranya, "A Hybrid Approach using Collaborative Filtering and Content Based Filtering for Recommender System," *J. Phys.: Conf. Ser.*, vol. 1000, no. 1, p. 012101, 2018.

[2] Z. Z. Darban and M. H. Valipour, "GHRS: Graph-based Hybrid Recommendation System with Application to Movie Recommendation," *Expert Syst. Appl.*, vol. 200, p. 116850, 2022.

[3] T. Qian, Y. Liang, and Q. Li, "Solving Cold Start Problem in Recommendation with Attribute Graph Neural Networks," *arXiv preprint arXiv:1912.12398*, 2019.

[4] H. Chen, Z. Wang, Y. Xu, X. Huang, and F. Huang, "GPatch: Patching Graph Neural Networks for Cold-Start Recommendations," *arXiv preprint arXiv:2209.12215*, 2022.

[5] C. Bernardis, M. F. Dacrema, and P. Cremonesi, "A Novel Graph-Based Model for Hybrid Recommendations in Cold-Start Scenarios," *arXiv preprint arXiv:1808.10664*, 2018.

[6] [Authors], "Hybrid Recommendation System Combined Content-Based Filtering and Collaborative Prediction Using Artificial Neural Network," *Simul. Model. Pract. Theory*, vol. 113, p. 102375, 2021.

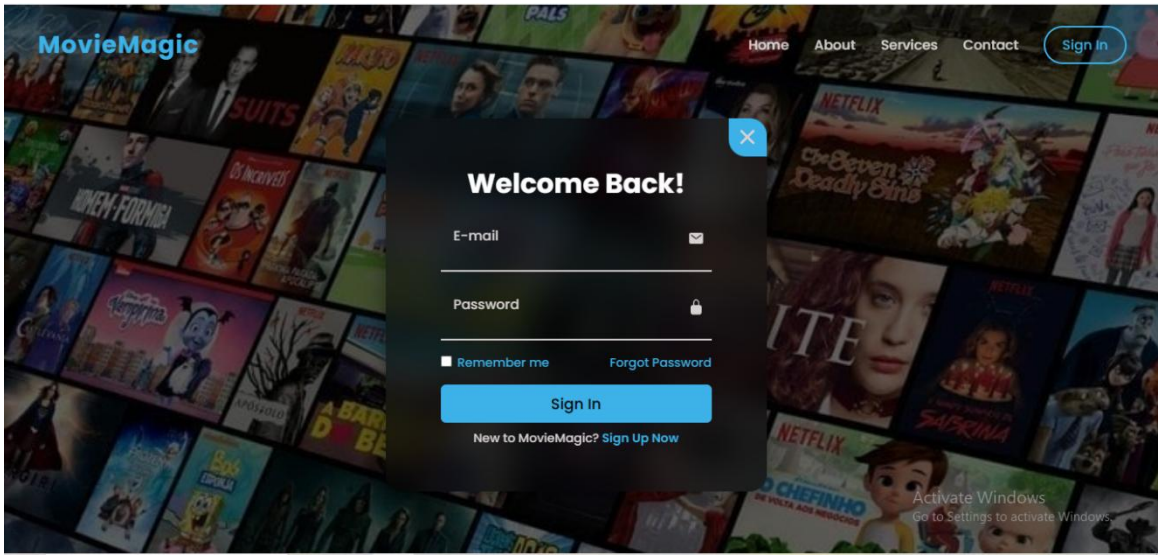
[7] R. Widayanti, M. H. R. Chakim, C. Lukita, U. Rahardja, and N. Lutfiani, "Improving Recommender Systems Using Hybrid Techniques of Collaborative Filtering and Content-Based Filtering," *J. Appl. Data Sci.*, vol. 5, no. 2, pp. 115–123, 2021.

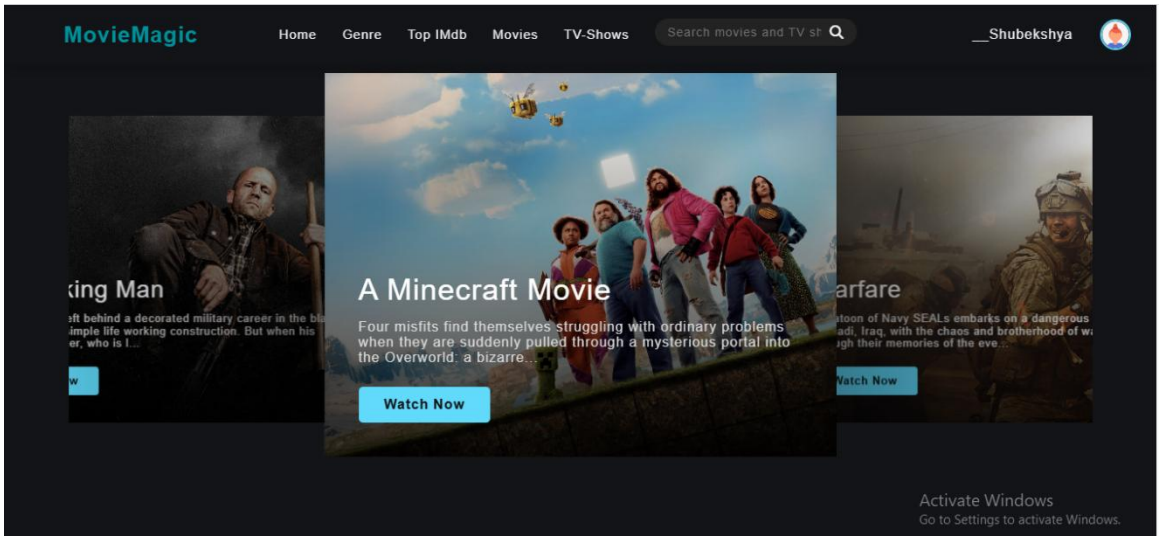
[8] Y. Li, "Hybrid Algorithm Based on Content and Collaborative Filtering in Recommendation System Optimization and Simulation," *Sci. Program.*, vol. 2021, p. 7427409, 2021.

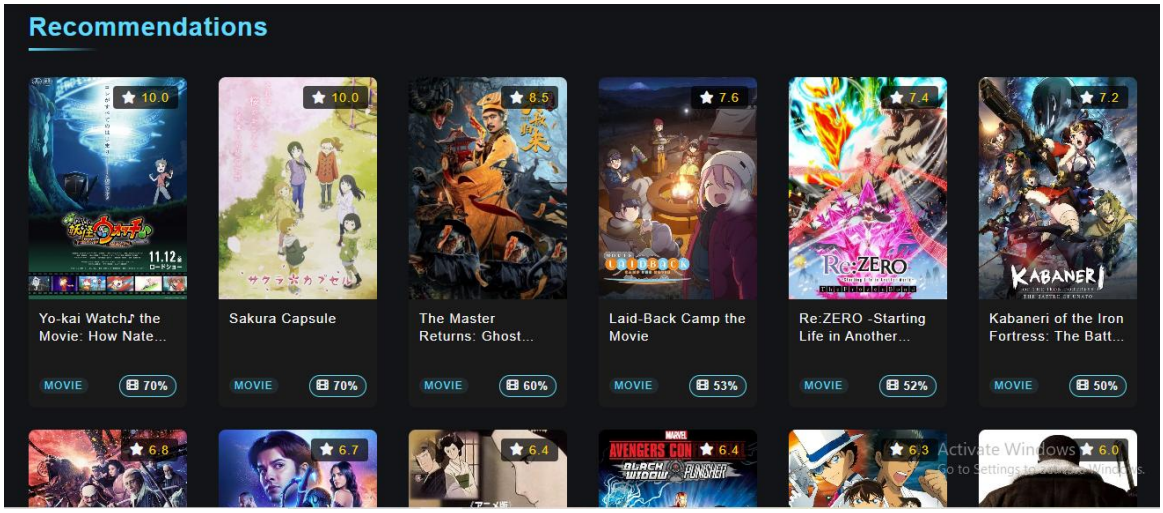
[9] FilmQuest. (2023). Open-source movie search and recommendation platform. GitHub Repository. Retrieved from https://github.com/username/filmquest

[10] MovieVault. (2024). Real-time movie discovery platform with Next.js 14. GitHub Repository. Retrieved from https://github.com/username/movievault

APPENDIX


MovieMagic
Home
About
Services
Contact
Sign In
Welcome Back!
E-mail
Password
Remember me
Forgot Password
Sign In
New to MovieMagic? Sign Up Now
Activate Windows
Go to Settings to activate Windows.


MovieMagic
Home
Genre
Top IMdb
Movies
TV-Shows
Search movies and TV sh
__Shubekshya
A Minecraft Movie
Four misfits find themselves struggling with ordinary problems when they are suddenly pulled through a mysterious portal into the Overworld: a bizarre...
Watch Now
Activate Windows
Go to Settings to activate Windows.


Recommendations
10.0
Yo-kai Watch♪ the Movie: How Nate...
MOVIE
70%
10.0
Sakura Capsule
MOVIE
70%
8.5
The Master Returns: Ghost...
MOVIE
60%
7.6
Laid-Back Camp the Movie
MOVIE
53%
7.4
Re:ZERO -Starting Life in Another...
MOVIE
52%
7.2
Kabaneri of the Iron Fortress: The Batt...
MOVIE
50%
6.8
6.7
6.4
6.4
6.3
6.0
Activate Windows
Go to Settings to activate Windows.

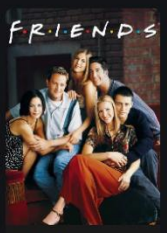
Trending in Your Area

Final Destination Bloodlines

MOVIE

Inside Out 2

MOVIE



Friends

TV

A Minecraft Movie

MOVIE

A Working Man

MOVIE

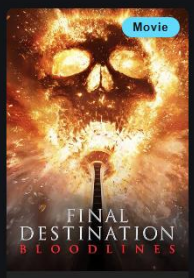
When Life Gives You Tangerines

TV

Your Watch History

Movies and shows you've watched

Clear History

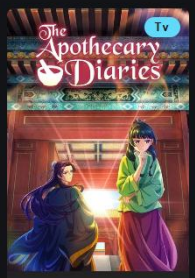


Final Destination ...

May 16, 2025

Havoc

May 8, 2025

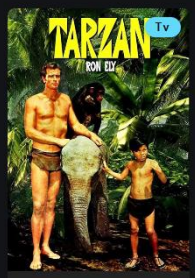


The Apothecary D...

May 8, 2025

Exterritorial

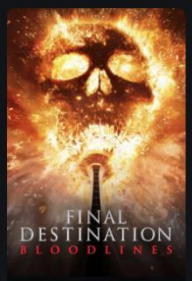
May 8, 2025



Tarzan

May 8, 2025

MY WATCHLIST



Final Destination Bloodlines

REMOVE

Havoc

REMOVE

The Siege

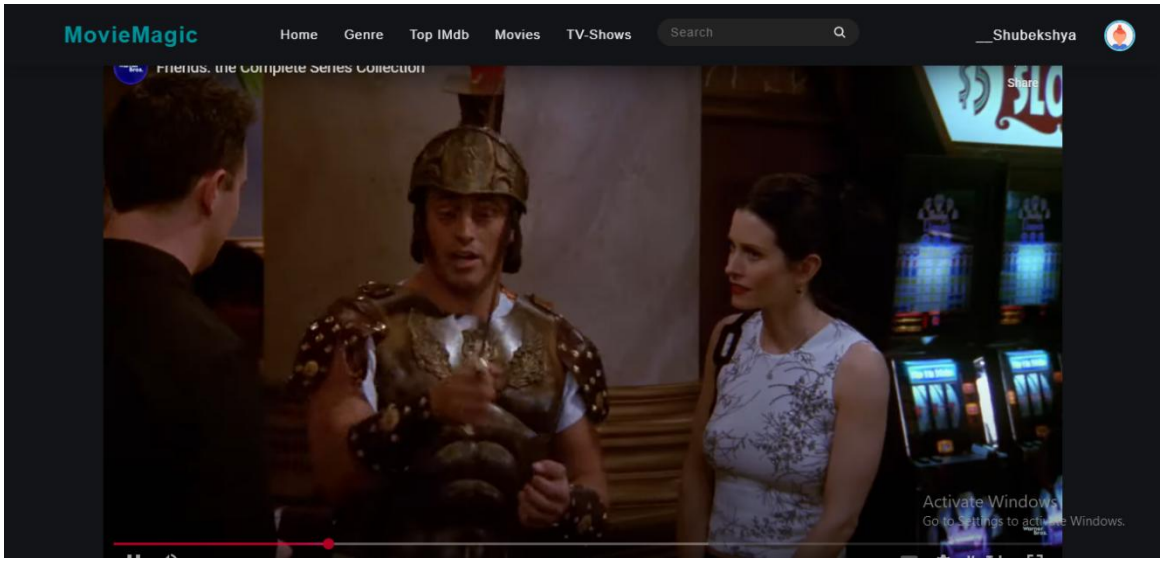
REMOVE

A Minecraft Movie

REMOVE

The Owl House

REMOVE



Reviews

Add Your Review

Your Review

Your Rating

Select rating (1 = Poor, 5 = Excellent)

Submit Review

__Shubekshya ★★★★★ May 17, 2025

Friends is a timeless sitcom filled with humor, heart, and unforgettable characters. The chemistry between the cast makes every episode a joy to watch. A perfect blend of comedy and emotion that still resonates today.